

Data Servers & Storage Systems

1 Fundamental Principles

Data-server systems (also known as **data storage systems**) differ fundamentally from transaction servers by shifting computation away from the server and onto the clients.

Data Shipping Architecture

Data shipping refers to an architecture where:

- Data items are sent from the server to the client
- All computation is performed at the client side
- Updated data is written back to the server

The server mainly acts as a **persistent storage** and **coordination point**.

1.1 Evolution of Data Units

- **Earlier Systems:** Operated on large units such as **pages** (containing many records) or entire objects.
- **Modern Systems:** Focus on **fine-grained data items**, improving flexibility and scalability.
- **Common Representations:** JSON, XML, or uninterpreted binary blobs.

Why Fine-Grained Data Matters

Smaller data units reduce unnecessary data transfer and improve concurrency at the cost of increased coordination.

2 The Latency Challenge

The main performance limitation in data-server systems arises from the vast difference between local memory access time and network communication delay.

Access Type	Typical Latency
Local Memory Access	< 100 ns
Network Round Trip	≈ 1 ms (10^6 ns)

Key Observation

Network communication is roughly **10,000× slower** than local memory access.

3 Client-Side Optimization Strategies

To hide or reduce network latency, clients employ several optimization techniques.

3.1 Prefetching

Prefetching involves proactively fetching data items before they are explicitly requested.

- Reduces future network round-trips
- Exploits access locality and prediction

3.2 Data Caching

Clients retain fetched data locally to reuse it across operations.

Cache Coherency

Cached data may persist **across transactions**. Before reuse, the client must ensure the data is still valid—often verified when requesting a lock from the server.

3.3 Lock Caching

Locks, like data, can also be cached at the client side.

- **Local Lock Acquisition:** Cached locks allow transactions to proceed without server communication.
- **Server Callback Mechanism:** If another client requests a conflicting lock, the server issues a callback requiring the lock to be returned.

4 Adaptive Lock Granularity

Lock granularity controls the trade-off between **communication overhead** and **concurrency**.

- **Lock Escalation:** Many fine-grained locks are replaced with a single coarse-grained lock to reduce overhead.
- **Lock De-escalation:** Coarse locks are split into finer locks when contention increases.

Design Insight

Adaptive granularity allows systems to maintain low overhead during low contention and high concurrency during heavy workloads.